Verified Compilers

Amir Ebrahim-zadeh

Announcement!

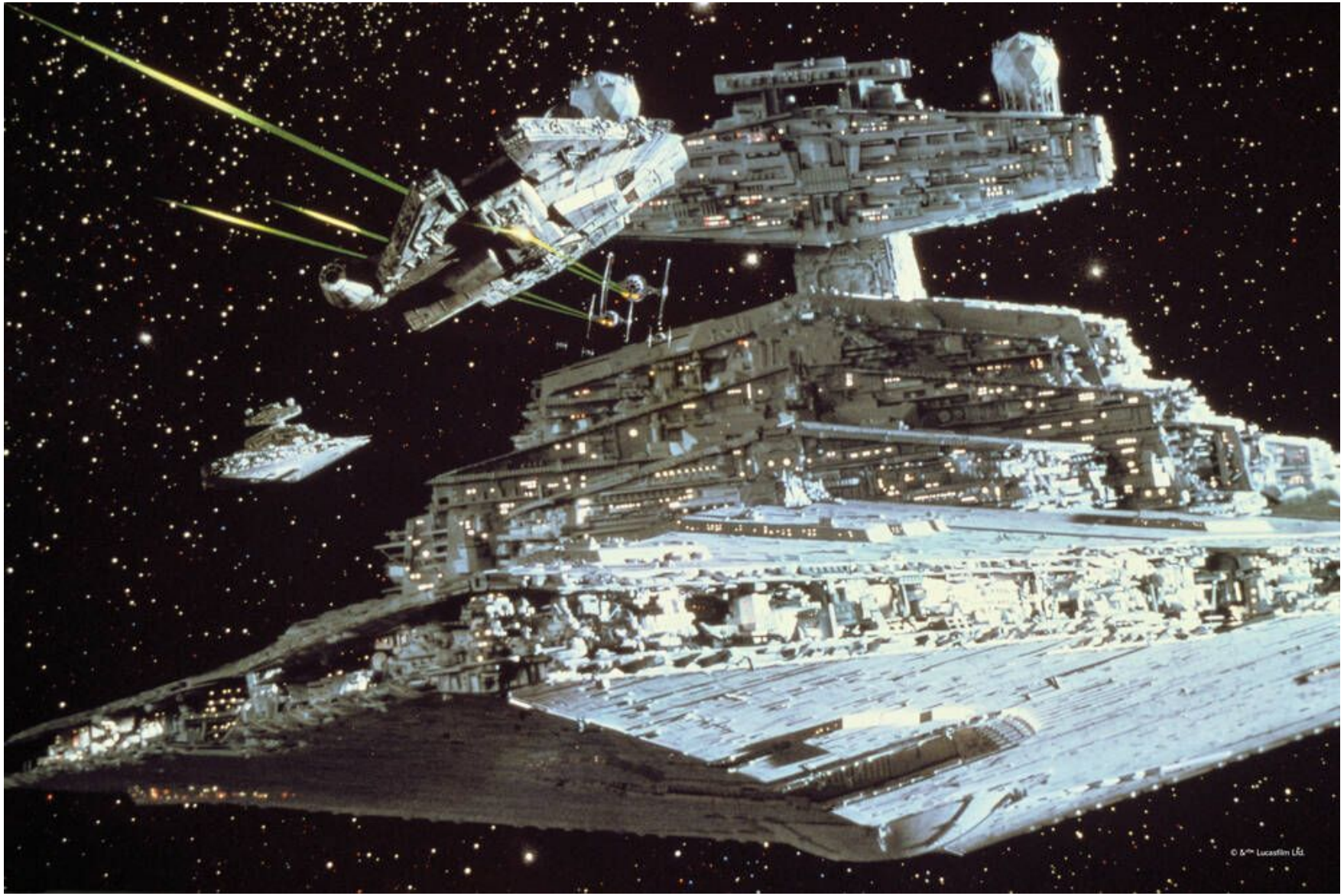
- I implemented a fully verified compiler for python with 93% performance compared to most efficient implementations
- Verified using Coq

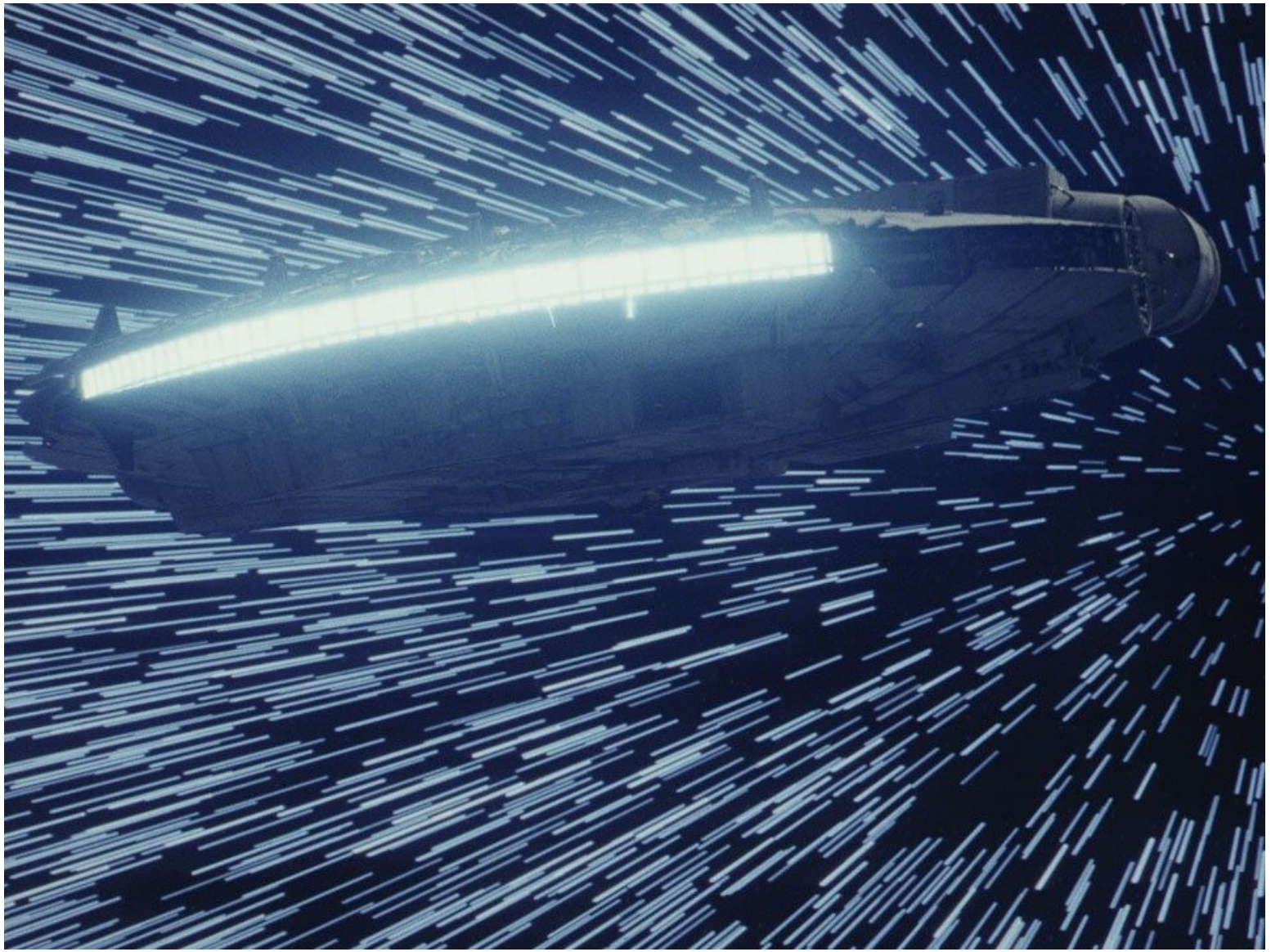
Announcement!

- I implemented a fully verified compiler for python with 93% performance compared to most efficient implementations
- Verified using Coq
- Download link: AprilFools.com


VERIFIED
COMPILATION

A long time ago in a program running a spaceship....





“Did you verify the hyperdrive code, Chewy?!”

“Yes…”

“But I compiled it using GCC”
Chewy responds…

“Safe” Compilation

- Using a safe compiler is essential for safety critical tasks
- Unsafe compilers can introduce bugs to safe programs

Core Property of interest: Semantic Preservation

- We need a new judgement: Observational behavior

Core Property of interest: Semantic Preservation

- We need a new judgement: Observational behavior
- Program S has behavior B

$$S \rightsquigarrow B$$

Observational Behavior

- I/O
- Observable memory locations
- Exit status

$$S \rightsquigarrow B$$

Core Property of interest: Semantic Preservation

- Strongest spec for a compiler translating source program S to compiled code C

$$\forall B,\ S \rightsquigarrow B \Leftrightarrow C \rightsquigarrow B$$

Methods

- Proofs via proof assistants (e.g. Coq)
- Automated proofs via SMT solvers

Roadmap

- The First Proof of Compiler Correctness
- CompCert: A Realistic Compiler
- CakeML: End-to-End Verified Compilation
- VeLLVM: Industrial Verified Compilation
- SMT-based Automatic Compiler Verification
- Type Preserving Compilation

The First Proof of Compiler Correctness

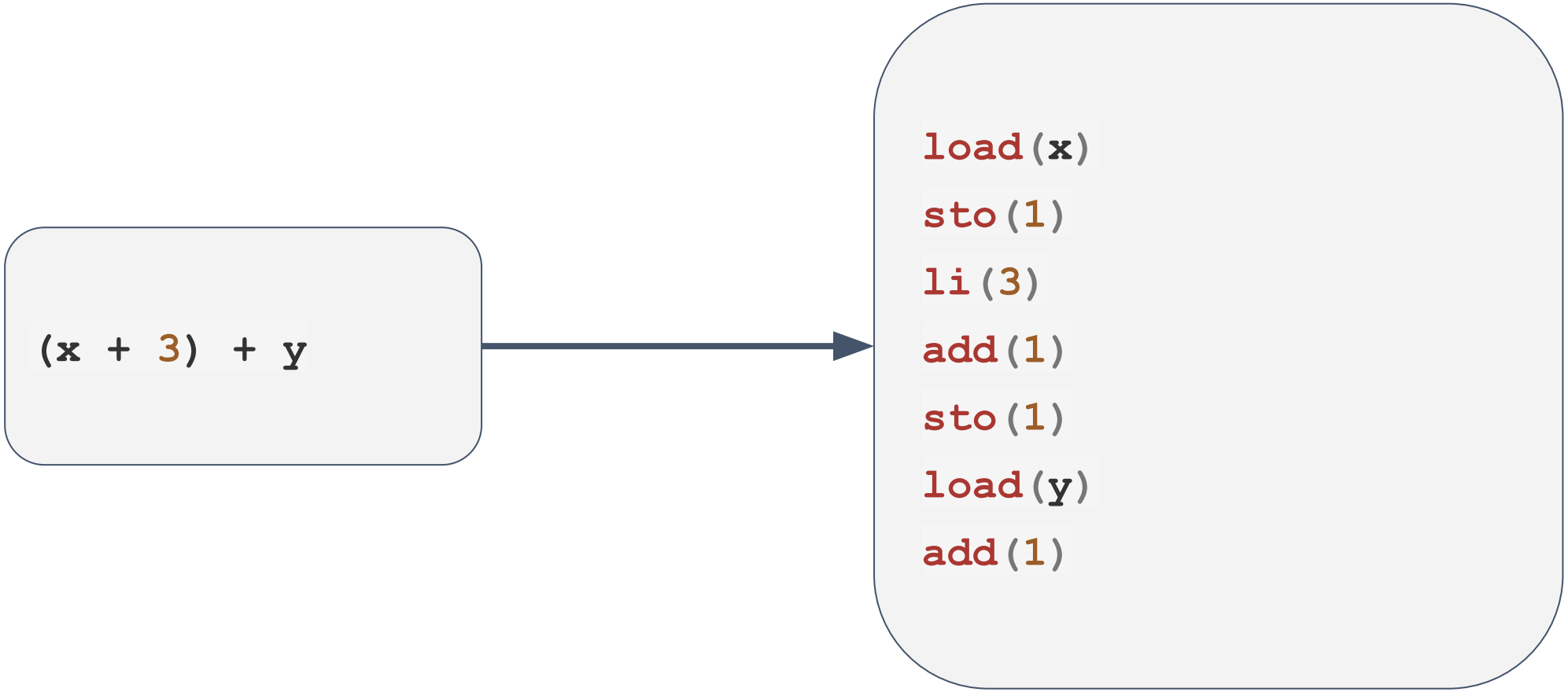
The First Proof of Compiler Correctness

- Correction of a Compiler for Arithmetic Expressions
- By John McCarthy and James Painter (1967)
- Problem: Compilers were trusted black boxes
- Solution: (Handwritten) proof of semantic preservation

A Compiler for Simple Arithmetics

- Input: Abstract syntax for a language of arithmetic expressions (constants, variables, and addition)
- Output: Machine code for an accumulator based architecture

Example: Simple Arithmetic Language



Correctness Theorem

- The value in the acc register must equal the expressions evaluation

$$value(e, \xi) = accumulator(execute(compile(e, t), \eta))$$

Limitation

- Toy language
- Proof not checked mechanically
- Mainly a proof of concept

CompCert: A Realistic Compiler

The Giant of Verified Compilation

- Formal Verification of a Realistic Compiler (2009)
- By Xavier Leroy

CompCert Motivation

- In safety critical software (e.g. aviation controller) bugs introduced by compilers are as deadly as a buggy input program.
- Compiles CLight to PowerPC

CompCert

- Coq for implementation and proof
- For source program S and target machine code C:

$$\forall B,\ S \rightsquigarrow B \Leftrightarrow C \rightsquigarrow B$$

CompCert

- Coq for implementation and proof
- For source program S and target machine code C:

$$\forall B,\ S \rightsquigarrow B \Leftrightarrow C \rightsquigarrow B$$

$$S \text{ is safe} \Rightarrow (\forall B,\ S \rightsquigarrow B \Leftrightarrow C \rightsquigarrow B)$$

CompCert

- Coq for implementation and proof
- For source program S and target machine code C:

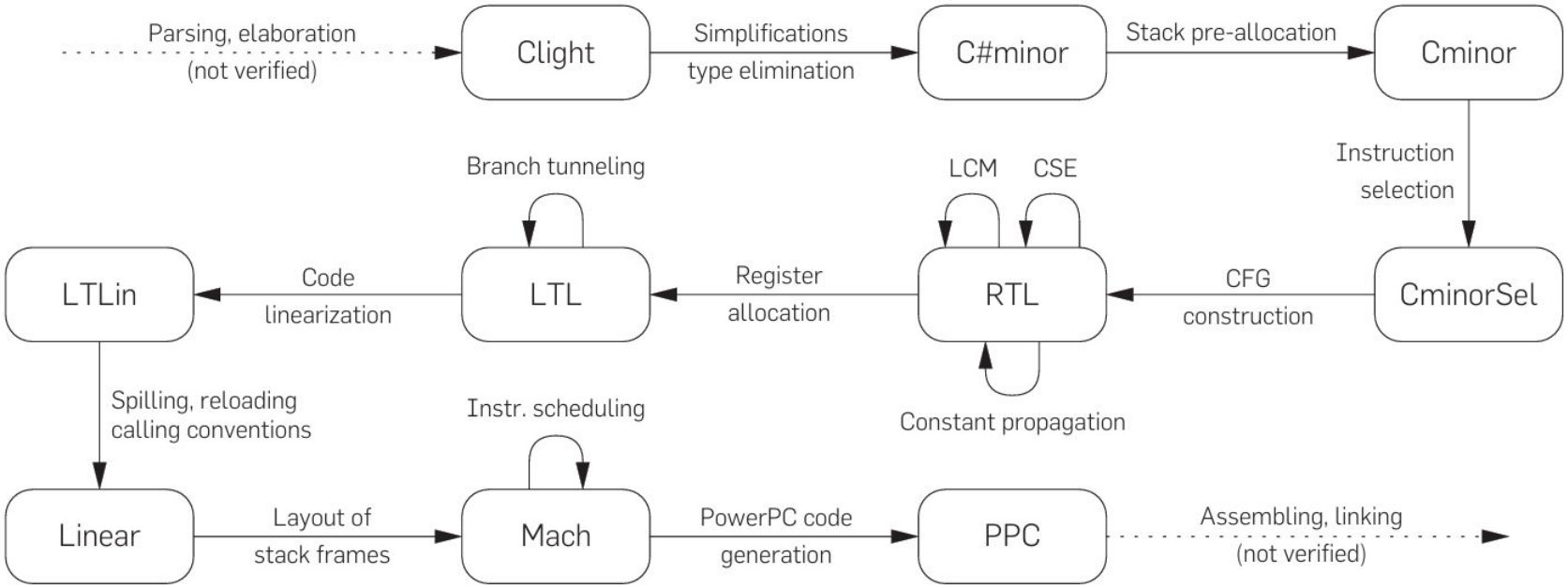
$$\forall B,\ S \rightsquigarrow B \Leftrightarrow C \rightsquigarrow B$$

$$S \text{ is safe} \Rightarrow (\forall B,\ S \rightsquigarrow B \Leftrightarrow C \rightsquigarrow B)$$

$$\text{Comp}(S) = \text{OK}(C) \Rightarrow S \approx C$$

CompCert Pipeline

- Three person-years of work
- 14 intermediate languages



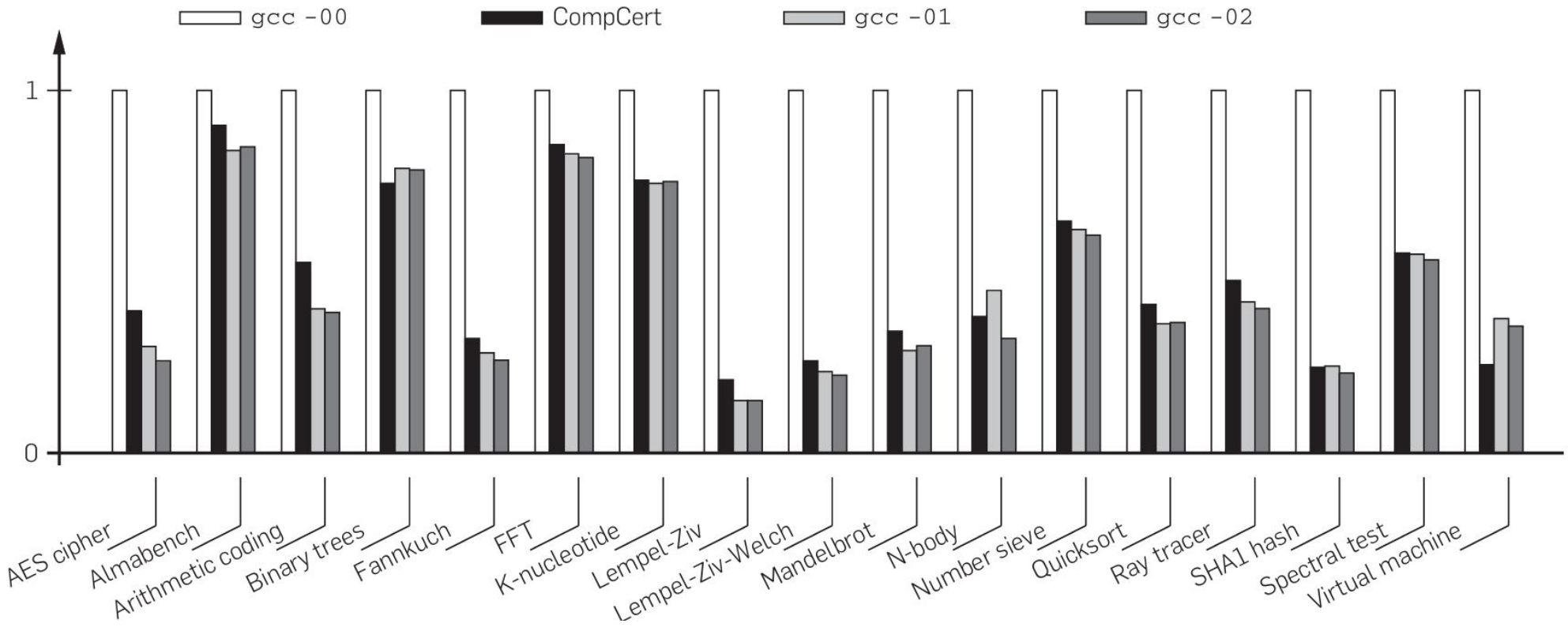
Compiler Verification vs Translation Validation

- Verifying optimization algorithms is difficult
- Translation Validation is usually easier

$$\forall S, C, \text{Validate}(S, C) = \text{true} \Rightarrow S \approx C$$

Performance

- 7% slower than `gcc -O1` and 12% slower than `gcc -O2`



Limitations and Open Problems

- CLight is small (Omits `goto` statements, `long long` types)
- Parser, assembler are completely unverified

CakeML:
End-to-End Verified Compilation

CakeML

- CakeML: End-to-End Verified Compilation (2014)
- Ramana Kumar et al.

CakeML

- Input language: CakeML, a subset of functional language Standard ML
- Compiled to: x86-64 machine code
- Verifying language: Entirely mechanized in HOL4 interactive theorem prover

End-to-End Verification

- Front-end Verified!
    - Formally verified Lexing and Parsing
    - Formally verified Type Checking
- Verified garbage collection!

Bootstrapping

- **The Meta-Problem**: If you write a verified compiler in ML, how do you compile it into machine code without trusting an unverified ML compiler?

Bootstrapping

- **The Meta-Problem**: If you write a verified compiler in ML, how do you compile it into machine code without trusting an unverified ML compiler?
- The CakeML compiler is written in CakeML itself!

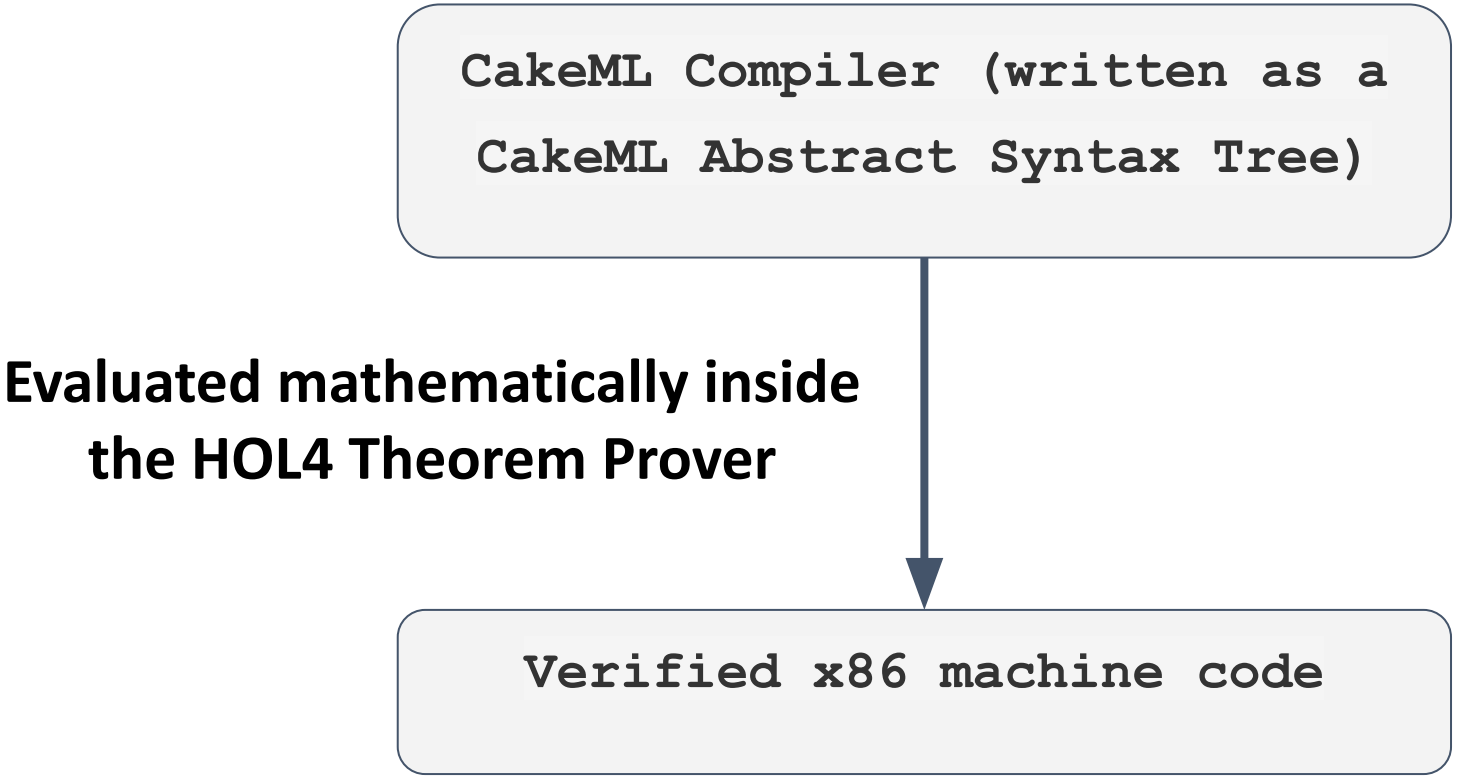
Bootstrapping

- **The Meta-Problem**: If you write a verified compiler in ML, how do you compile it into machine code without trusting an unverified ML compiler?
- The CakeML compiler is written in CakeML itself!
- They use the HOL4 theorem prover evaluate the compiler on its own source code.

Bootstrapping

- **The Meta-Problem**: If you write a verified compiler in ML, how do you compile it into machine code without trusting an unverified ML compiler?
- The CakeML compiler is written in CakeML itself!
- They use the HOL4 theorem prover evaluate the compiler on its own source code.
- HOL4 is translated to CakeML by a synthesizer, easy due to syntax similarity

Bootstrapping



Bootstrapping

- Both steps are verified:

$$\mathrm{Synth}(\mathrm{CakeML\_HOL}) = \mathrm{CakeML\_CakeML}$$

$$\mathrm{CakeML\_HOL}(\mathrm{CakeML\_CakeML}) = \mathrm{CakeML\_x86}$$

- Use $\mathrm{CakeML\_x86}$ from that point

VeLLVM:
A Verified Industrial Compiler

VeLLVM: A Pragmatic Shift

- Formalizing the LLVM intermediate representation for verified program transformations
- Jianzhou Zhao et al. (2012)

LLVM Intermediate Representation

- **Infrastructure**: LLVM is the massive, highly optimized C++ compiler framework
- **Intermediate Representation (IR)**: A strictly typed, RISC-like, Static Single Assignment (SSA) language
- Allows adding optimization passes in the IR level

Verified LLVM

- CompCert and CakeML require abandoning existing compilers
- **The Problem**: How to bring formal safety to LLVM, the massive C++ infrastructure?
- **The Challenge**: The IR is notoriously complex, featuring intentional undefined behaviors
- **The Solution**: VeLLVM provides a formal semantics of LLVM IR in Coq

Verified LLVM

- Translation Validation: Provides a Coq-verified checker that runs after an unverified LLVM optimization
- Proves the optimized program is semantically equivalent to the unoptimized program after each step

SMT-based Automatic Compiler Verification

Alive

- Provably Correct Peephole Optimizations with Alive
- Nuno Lopes et al. (2015)
- Introduces a new paradigm for Verified compilation

What is a “Peephole” Optimization? :)

- A micro-optimization performed locally
- Typical example: Converting an XOR and an ADD into a single SUB

$$(x \oplus -1) + C \Rightarrow (C - 1) - x$$

The Problem

- The InstCombine pass grew to over 15,000 lines of complex pattern-matching messy code
- The single buggiest file in the LLVM framework

The Solution: Alive

- A DSL to write peephole optimization
- Equipped with an SMT solver to automatically verify correctness of the optimization
- A synthesizer automatically translates the Alive code to C++
- Used to translate and verify InstCombine
- Alive provides counterexamples to incorrect rules

Example Optimization

```
// Source Template
%1 = xor %x, -1
%2 = add %1, C
  =>
// Target Template
%1 = sub C-1, %x
```

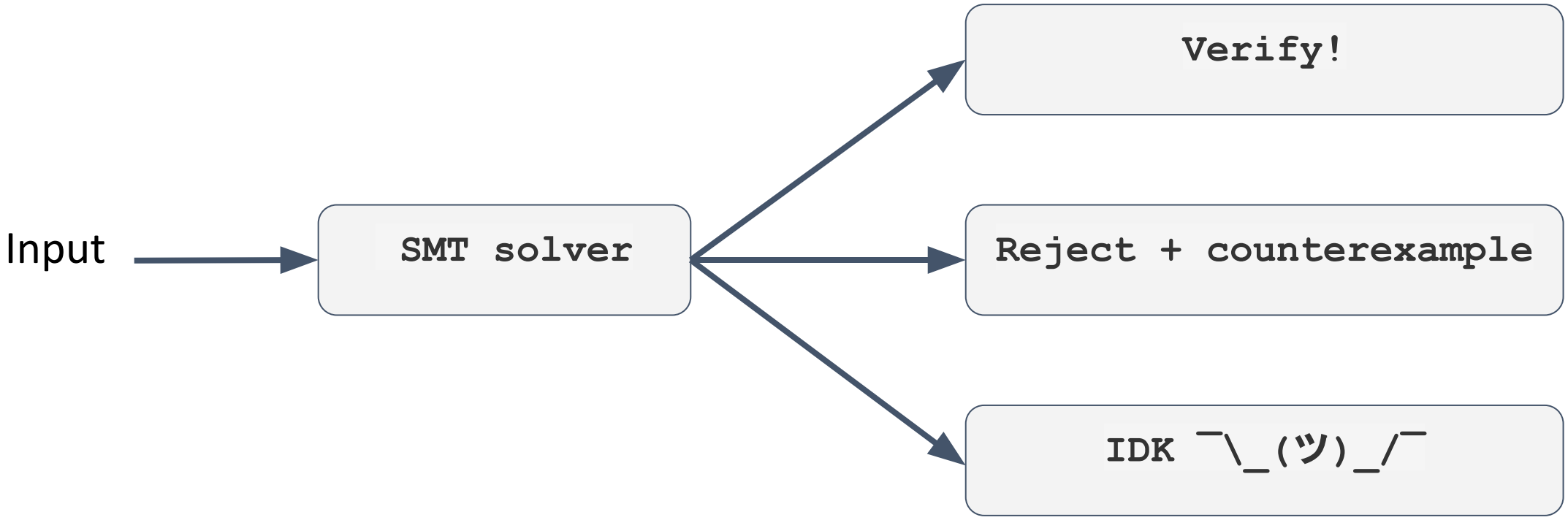
Contributions

- Translated 334 optimizations into Alive
- Discovered 8 bugs in the LLVM compiler!
- Alive introduced zero performance overhead to the compiled binaries compared to the hand-written C++

Major Limitations

- Unlike Dafny, Alive does not provide mechanism for proof hints (assertions, lemmas, etc)
- Struggle with non-linear arithmetic (e.g. division, mod)

SMT Otput



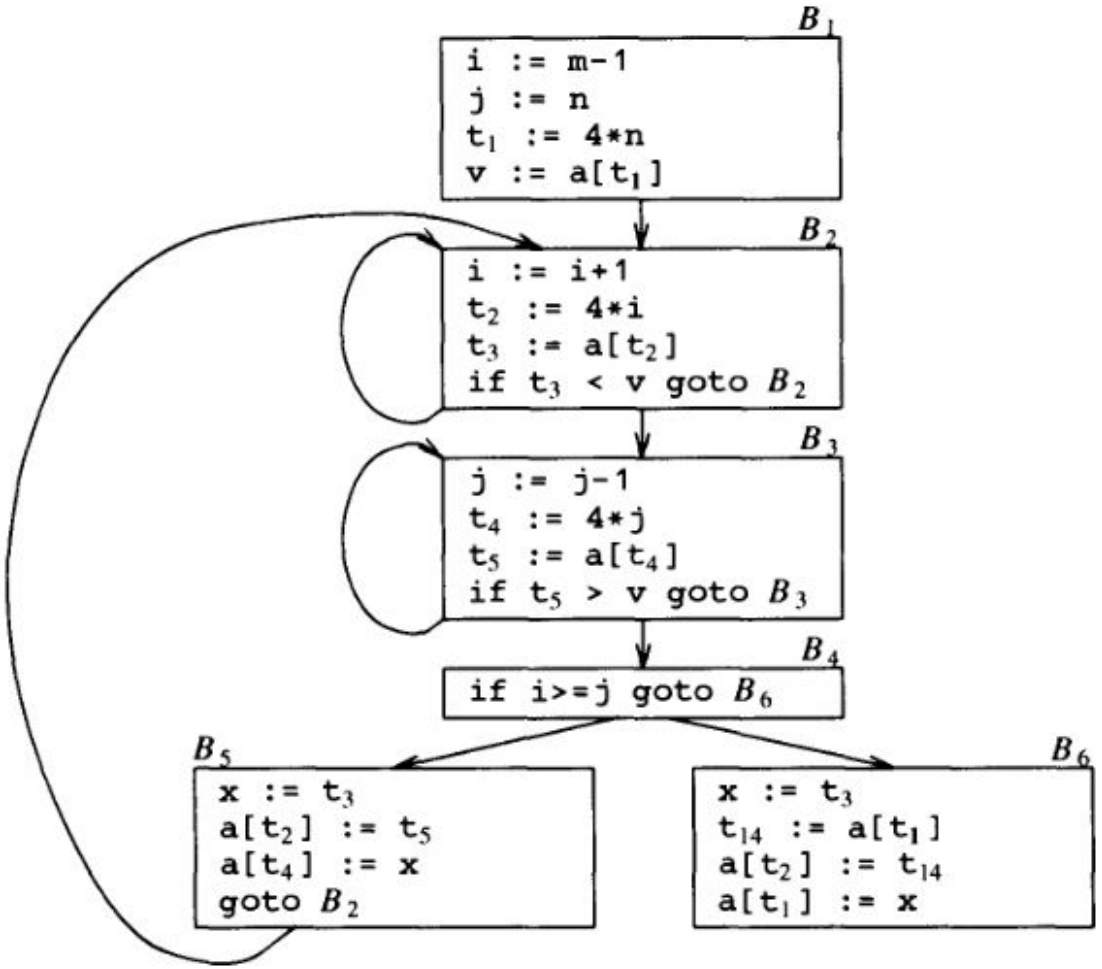
Major Limitations

- Unlike Dafny, Alive does not provide mechanism for proof hints (assertions, lemmas, etc)
- Struggle with non-linear arithmetic (e.g. division, mod)
- Possible solution: Extend Alive to accept proof hints

Major Limitations Cont.

- Peephole optimizations are local, unable to perform global optimizations

Global vs Local Optimization



Major Limitations Cont.

- Peephole optimizations are local, unable to perform global optimizations
- Can be generalized and equipped with user provided invariants

Type Preserving Compilation

Type Safe Compilation

- From System F to Typed Assembly Language
- Morrisett, Walker, Crary, & Glew (1999)

Type Safe Compilation

- **The Problem**: How do you safely execute downloaded, untrusted code (like web applets or plugins) without trusting the compiler that built it?

Typed Assembly Language

- **The TAL Solution**: Push the type system all the way down to the metal

```
l_main:
  code[]{}.                          % entry point
    mov r1,6
    jmp l_fact
l_fact:
  code[]{r1:int}.                    % compute factorial of r1
    mov r2,r1                        % set up for loop
    mov r1,1
    jmp l_loop
l_loop:
  code[]{r1:int,r2:int}.             % r1: the product so far,
                                     % r2: the next number to be multiplied
    bnz r2,l_nonzero                 % branch if not zero
    halt[int]                        % halt with result in r1
l_nonzero:
  code[]{r1:int,r2:int}.
    mul r1,r1,r2                     % multiply next number
    sub r2,r2,1                      % decrement the counter
    jmp l_loop
```

Typed Assembly Language

- TAL augments standard RISC assembly with type annotations for registers, memory locations, and code
- The Type Checker: Before the host machine runs the assembly code, it runs a type-checker over the instructions
- The Guarantee: If the assembly code type-checks, it is guaranteed to be memory-safe and control-flow safe. It cannot crash or access forbidden memory.

The compiler

- Input: System F is the input language
- The types are preserved to the machine code level

Impact

- We no longer need to verify the compiler
- Logical bugs are permitted, crashes or exploits not

Safe Compilation Matters!

- The First Proof of Compiler Correctness
- CompCert: A Realistic Compiler
- CakeML: End-to-End Verified Compilation
- VeLLVM: Industrial Verified Compilation
- SMT-based Automatic Compiler Verification
- Type Preserving Compilation

Questions?